Учебник по Qt для мобильных устройств

Nov 9, 2026 · @Алексей

Этот учебник для тех, кто никогда ничего не делал для телефонов. Читай по порядку, не прыгай. Каждый урок — маленький шаг, в конце — простое задание «сделай сам».

Словарик (загляни сюда, если встретил непонятное слово)

Слово	Что это значит по-человечески
Qt (читается «кьют»)	Набор готовых деталей, из которых собирают программы. Как конструктор LEGO для программистов.
Приложение	Программа на телефоне. Иконка, на которую тыкаешь.
Android / iOS	Две «системы» телефонов. Android — Samsung, Xiaomi и т. д. iOS — это iPhone.
QML	Язык, на котором описываешь, *как выглядит экран*: где кнопка, какого цвета, что написано.
C++	Язык, на котором пишешь *как программа думает*: считает, сохраняет, качает из интернета.
Qt Creator	Программа на компьютере, где пишешь код и жмёшь «Запустить». Как Word, только для кода.
Сборка	Превращение твоего текста-кода в настоящее приложение, которое ставится на телефон.
APK	Файл приложения для Android. Как установщик `.exe` на компьютере.
SDK / NDK	Инструменты от Google, без них Android-приложение не собрать. Просто ставишь и забываешь.

Слово	Что это значит по-человечески
Kit	Настройка «для какого телефона собираем». Выбираешь из списка.
Свойство	Характеристика предмета: ширина, цвет, текст. Как «цвет машины — красный».
Сигнал	Сообщение «что-то случилось». Например, «на кнопку нажали».

> Не надо всё запоминать сразу. Вернёшься к таблице, когда понадобится.

Урок 1. Что такое Qt — на пальцах

Qt — это способ написать приложение **один раз** и потом запустить его **и на Android, и на iPhone**. Не надо делать две разные программы.

Пример из жизни. Ты строишь дом. Можно каждый кирпич лепить самому — долго. А можно купить готовые кирпичи, окна и двери. Qt — это магазин готовых деталей: кнопки, списки, окна, работа с камерой и интернетом уже сделаны за тебя.

Приложение состоит из двух частей:

1. **Лицо (QML)** — то, что видит человек: кнопки, текст, картинки.
2. **Мозги (C++)** — то, что скрыто: подсчёты, сохранение, интернет.

Как в кафе: зал с меню — это QML, кухня — это C++. Клиент кухню не видит, но без неё еды не будет.

Хорошая новость: маленькое приложение можно сделать почти целиком на QML. C++ понадобится позже, в уроке 7.

Что нужно, чтобы начать:

- Компьютер (Windows, Mac или Linux). Для iPhone — только Mac, таковы правила Apple.
- Телефон и USB-кабель (или виртуальный телефон на компьютере — о нём в уроке 2).
- Около 20 ГБ свободного места на диске.
- Час-два на установку в первый раз.

> **Сделай сам:** запиши на бумажке, какое приложение хочешь сделать, и раздели его на «лицо» (что на экране) и «мозги» (что оно делает внутри). Например, для крестиков-ноликов: лицо — поле 3×3, мозги — проверка, кто выиграл.

Урок 2. Ставим программы — по кнопкам

Нужно поставить три вещи: **Android Studio** (инструменты Google), **Qt** с **Qt Creator** и связать их между собой. Делай строго по порядку.

Шаг 1. Android Studio

1. Открой сайт developer.android.com/studio и скачай Android Studio.
2. Установи, везде жми «Далее».
3. Запусти. Она сама докачает нужное — подожди.
4. На стартовом экране нажми **More Actions → SDK Manager**.
5. Во вкладке **SDK Tools** поставь галочки: *NDK (Side by side)*, *Android SDK Command-line Tools*, *Android SDK Platform-Tools*. Нажми **OK**.

Шаг 2. Qt и Qt Creator

1. Открой qt.io/download-open-source, скачай **Qt Online Installer**.
2. Запусти, создай бесплатный аккаунт Qt (почта + пароль).
3. Когда спросит, что ставить, выбери **Custom installation**.
4. Раскрой самую свежую версию **Qt 6.x** и отметь:
   - **Android** (все пункты внутри);
   - сборку для своего компьютера (*MinGW* на Windows, *macOS* на Mac) — чтобы быстро проверять на компьютере;
   - **iOS** — только если у тебя Mac.
5. Жми «Далее» до конца. Качаться будет долго — это нормально.

Шаг 3. Связываем Qt Creator с Android

1. Открой **Qt Creator**.
2. Меню **Edit → Preferences** (на Mac: **Qt Creator → Settings**).
3. Слева **Devices**, сверху вкладка **Android**.
4. В строке *JDK location* и *Android SDK location* жми кнопки поиска/скачивания рядом. Если появится кнопка **Set Up SDK** — жми её.

5. Жди, пока все строчки внизу станут с **зелёными галочками**. Зелёные = всё хорошо. Красные = чего-то не хватает, читай надпись рядом.

Шаг 4. Готовим телефон

Если есть Android-телефон:

1. Настройки → О телефоне → 7 раз тыкни на **«Номер сборки»**. Появится «Вы стали разработчиком».
2. Настройки → Для разработчиков → включи **«Отладка по USB»**.
3. Подключи кабель. На телефоне выскочит вопрос — жми **«Разрешить»**.

Если телефона нет — сделай виртуальный: в Android Studio **More Actions → Virtual Device Manager → Create Device**, выбери любой Pixel, дальше «Далее». Получится телефон в окошке на экране компьютера.

Для iPhone (только на Mac): поставь **Xcode** из App Store, один раз запусти его и согласись с лицензией. В Xcode → Settings → Accounts добавь свой Apple ID.

Если что-то не так

Что видишь	Что делать
Красные крестики в настройках Android	Нажми Set Up SDK, перезапусти Qt Creator
Телефон не появляется	Другой кабель (многие только заряжают), проверь «Отладка по USB»
«NDK version mismatch»	В SDK Manager поставь версию NDK, которую просит Qt

> **Сделай сам:** добейся, чтобы в Preferences → Devices → Android всё было зелёным, а твой телефон появился во вкладке Devices. Без этого дальше не иди.

Урок 3. Первое приложение: кнопка-счётчик

Сделаем приложение: на экране число и кнопка. Жмёшь кнопку — число увеличивается на 1.

Шаг 1. Создаём проект

1. Qt Creator → **File → New Project**.
2. Выбери **Qt Quick Application** → **Choose**.
3. Имя: `counter` (только латиница, без пробелов). Папка — тоже без русских букв и пробелов (например, `C:\qt_projects`). Это важно, иначе сборка может сломаться.
4. Далее, Далее… На экране **Kits** отметь **Android** и свой компьютер (Desktop).
5. **Finish**.

Qt Creator создаст папку с файлами:

Файл	Зачем он	Надо ли трогать
`Main.qml`	Как выглядит экран	**Да**, тут вся работа
`main.cpp`	Запускает приложение	Пока нет
`CMakeLists.txt`	Список файлов и настроек сборки	Только когда добавляешь новые файлы

Шаг 2. Пишем код

Открой `Main.qml`, удали всё и вставь:

```
import QtQuick
import QtQuick.Controls

// ApplicationWindow — это всё окно приложения
ApplicationWindow {
    visible: true          // показать окно
    width: 360             // ширина (на телефоне игнорируется — будет весь
экран)
    height: 640            // высота
    title: "Счётчик"

    // наша «переменная» — коробочка, в которой лежит число
    property int count: 0

    // Column ставит элементы друг под другом
    Column {
        anchors.centerIn: parent   // по центру экрана
        spacing: 20                // расстояние между ними

        Label {
```

```
            text: count               // показываем число
            font.pixelSize: 64        // крупно
            anchors.horizontalCenter: parent.horizontalCenter
        }

        Button {
            text: "Плюс один"
            onClicked: count = count + 1   // что делать при нажатии
        }

        Button {
            text: "Сбросить"
            onClicked: count = 0
        }
    }
}
```

Разбор по строчкам:

- `import ...` — «достать из коробки детали». Без них не будет кнопок.
- `Имя { ... }` — создать предмет. Внутри фигурных скобок — его настройки и то, что лежит внутри.
- `настройка: значение` — задать свойство. `width: 360` = «ширина 360».
- `onClicked:` — «когда нажали, сделай вот это».
- `//` — комментарий, программа его не читает. Пиши туда заметки для себя.

Волшебство: мы нигде не писали «обнови текст». `text: count` — это не «один раз скопировать», а «всегда показывать то, что сейчас в count». Это называется **привязка**. Поменялось число — текст поменялся сам.

Шаг 3. Запускаем

1. **Сначала на компьютере:** слева внизу есть значок монитора — выбери там *Desktop*, нажми зелёный ▶ (или Ctrl+R). Откроется окно с кнопками. Потыкай.
2. **Теперь на телефоне:** в том же месте выбери *Android* и свой телефон, снова ▶. Первая сборка идёт несколько минут. Потом приложение само откроется на телефоне.

> Совет на всю жизнь: проверяй всё на компьютере (это быстро), а на телефон ставь, когда нужно проверить на настоящем устройстве.

Ошибки новичка

Ошибка	В чём дело
Белый экран, ничего нет	Забыл `visible: true`
`Button is not a type`	Забыл `import QtQuick.Controls`
Красная строчка в редакторе	Пропущена скобка `}` или двоеточие
Текст вместо числа не меняется	Написал `text: "count"` в кавычках — это просто слово, а не переменная

> **Сделай сам:** добавь кнопку «Минус один». Подсказка: скопируй кнопку «Плюс один» и поменяй два символа.

Урок 4. QML — как «рисовать» экран

В QML ты не рисуешь мышкой, а *описываешь словами*: «тут синий прямоугольник, в нём белая надпись». Всё строится как матрёшка: предмет внутри предмета.

4.1. Базовые кирпичики

Элемент	Что это	Пример
`Rectangle`	Прямоугольник (фон, карточка)	`Rectangle { width: 100; height: 50; color: "red" }`
`Text` / `Label`	Надпись	`Label { text: "Привет" }`
`Image`	Картинка	`Image { source: "cat.png" }`
`Button`	Кнопка	`Button { text: "OK" }`
`TextField`	Поле, куда пишут текст	`TextField { placeholderText: "Имя" }`
`Switch` / `CheckBox`	Выключатель / галочка	`Switch { text: "Звук" }`
`Slider`	Ползунок	`Slider { from: 0; to: 100 }`
`ListView`	Список, который листается	см. 4.6

4.2. id — имя предмета

Чтобы один предмет мог обратиться к другому, дай ему имя через `id`. Как бейджик с именем.

```
Column {
    TextField { id: nameField; placeholderText: "Как тебя зовут?" }
    Label { text: "Привет, " + nameField.text + "!" }
}
```

Пишешь имя — надпись меняется на лету. Снова привязка.

4.3. Карточка — матрёшка из предметов

```
Rectangle {                     // сама карточка
    width: 300; height: 100
    radius: 16                  // скруглённые углы
    color: "#4a90e2"            // цвет можно словом или кодом

    Row {                       // внутри — строка
        anchors.centerIn: parent
        spacing: 12
        Image { source: "avatar.png"; width: 48; height: 48 }
        Column {
            Label { text: "Вася Пупкин"; color: "white"; font.bold: true }
            Label { text: "в сети"; color: "white" }
        }
    }
}
```

`Row` — ставит предметы **в строчку**, `Column` — **столбиком**. Из этих двух собирается почти любой экран.

4.4. Реакция на действия: on…

У каждого предмета есть события. Чтобы на них ответить, пиши `on` + название с большой буквы:

```
Button { text: "Нажми"; onClicked: console.log("нажали") }
Switch { onCheckedChanged: console.log("включён?", checked) }
TextField { onAccepted: console.log("ввели:", text) }   // нажали Enter
Slider { onMoved: console.log("значение:", value) }
```

`console.log(...)` пишет текст в панель внизу Qt Creator (**Application Output**). Это твой главный способ понять, что происходит.

Если действий несколько — бери в фигурные скобки:

```
Button {
    text: "Войти"
    onClicked: {
        if (nameField.text === "") {
            errorLabel.text = "Введи имя!"
        } else {
            errorLabel.text = ""
            console.log("Вошёл:", nameField.text)
        }
    }
}
```

4.5. Свои кнопки в отдельном файле

Если одинаковая штука нужна много раз — вынеси её в файл. **Имя файла с большой буквы = новый предмет.**

Создай файл `BigButton.qml` (правый клик по проекту → Add New → QML File):

```
import QtQuick
import QtQuick.Controls

Button {
    id: root
    property color bg: "#4a90e2"    // своя настройка «цвет фона»
    implicitWidth: 240
    implicitHeight: 56
    background: Rectangle { color: root.down ? Qt.darker(root.bg) : root.bg;
radius: 12 }
    contentItem: Label {
        text: root.text; color: "white"; font.pixelSize: 18
        horizontalAlignment: Text.AlignHCenter; verticalAlignment:
Text.AlignVCenter
    }
}
```

Теперь в `Main.qml` можно писать:

```
BigButton { text: "Играть" }
BigButton { text: "Выйти"; bg: "crimson" }
```

Важно: допиши новый файл в `CMakeLists.txt` в строчку `QML_FILES Main.qml BigButton.qml`. Иначе на телефоне он не найдётся.

4.6. Списки

```
ListView {
    anchors.fill: parent
    model: ["Яблоки", "Хлеб", "Молоко", "Сыр"]   // что показывать
    delegate: ItemDelegate {                          // как выглядит одна
строка
        width: ListView.view.width
        text: modelData                               // текущий элемент
        onClicked: console.log("Выбрали:", modelData)
    }
}
```

- **model** — *что* показывать (данные).
- **delegate** — *как* показывать одну строку (шаблон). Как формочка для печенья: одна формочка, много печенек.

Список, в который можно добавлять и удалять, делается через `ListModel`:

```
ListModel { id: shopping }

TextField { id: input; onAccepted: { shopping.append({ name: text }); text =
"" } }

ListView {
    model: shopping
    delegate: ItemDelegate {
        text: name
        onPressAndHold: shopping.remove(index)   // долго держишь — удаляется
    }
}
```

4.7. Анимация в одну строчку

```
Rectangle {
    id: ball
    width: 60; height: 60; radius: 30; color: "orange"
    Behavior on x { NumberAnimation { duration: 300 } }   // двигаться плавно
    TapHandler { onTapped: ball.x = ball.x + 100 }
}
```

`Behavior on x` означает: «когда меняется x — не прыгай, а едь плавно».

> **Сделай сам:** список покупок из 4.6, вверху — поле ввода и кнопка «Добавить» (своя `BigButton`). Всё в одном `Column`.

Урок 5. Чтобы на любом телефоне выглядело нормально

Телефоны бывают узкие, широкие, с вырезом под камеру, их поворачивают боком. **Главное правило: не пиши точные цифры для размеров экрана.** Вместо «ширина 400» говори «во всю ширину» или «половина экрана».

5.1. Якоря — «прилепи к краю»

Представь, что предмет приклеен скотчем к краям экрана или другого предмета.

```
// верхняя полоска на всю ширину
Rectangle {
    id: topBar
    anchors.left: parent.left       // левый край — к левому краю экрана
    anchors.right: parent.right     // правый — к правому
    anchors.top: parent.top         // верх — к верху
    height: 56
    color: "steelblue"
}

// всё остальное место под полоской
Rectangle {
    anchors.top: topBar.bottom      // верх — к НИЗУ полоски
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.bottom: parent.bottom
    color: "whitesmoke"
}
```

Шпаргалка:

Написать	Значит
`anchors.fill: parent`	Занять всё место родителя
`anchors.centerIn: parent`	В центр
`anchors.margins: 16`	Отступ 16 со всех сторон
`width: parent.width / 2`	Половина ширины родителя

5.2. Layouts — «расставь сам»

Когда предметов много, проще поручить расстановку «расставлятору». Добавь `import QtQuick.Layouts` и используй `ColumnLayout` / `RowLayout`. Их суперсила — «растянись на свободное место».

```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts

ApplicationWindow {
    visible: true
    width: 360; height: 640

    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 16
        spacing: 12

        Label { text: "Вход"; font.pixelSize: 28 }

        TextField {
            Layout.fillWidth: true          // растянуть по ширине
            placeholderText: "Почта"
        }
        TextField {
            Layout.fillWidth: true
            placeholderText: "Пароль"
            echoMode: TextInput.Password    // точки вместо букв
        }

        Item { Layout.fillHeight: true }    // «пружина»: выталкивает кнопку
вниз

        Button {
```

```
            Layout.fillWidth: true
            text: "Войти"
        }
    }
}
```

Хитрость с **пустым `Item` и `fillHeight`** — это невидимая пружина. Она занимает всё лишнее место, и кнопка оказывается внизу на любом телефоне.

5.3. Телефон повернули боком

```
ApplicationWindow {
    id: window
    // true, если ширина больше высоты (телефон лежит боком)
    readonly property bool lying: width > height

    GridLayout {
        anchors.fill: parent
        columns: window.lying ? 2 : 1     // боком — две колонки, стоя — одна
        Rectangle { Layout.fillWidth: true; Layout.fillHeight: true; color:
"tomato" }
        Rectangle { Layout.fillWidth: true; Layout.fillHeight: true; color:
"gold" }
    }
}
```

`условие ? если_да : если_нет` — короткая запись «если… то… иначе…».

5.4. Размеры пальца

- Qt сам пересчитывает размеры под чёткость экрана. `48` на дешёвом и дорогом телефоне будет примерно одного размера в сантиметрах.
- Кнопка должна быть не меньше **48 на 48** — иначе в неё не попасть пальцем.
- Шрифт основного текста — около **16**, заголовков — **22–28**.

5.5. Касания и жесты

К любому предмету можно прицепить «ловитель жестов»:

Хочу	Пишу внутрь предмета
Тыкнули	`TapHandler { onTapped: ... }`
Два раза тыкнули	`TapHandler { onDoubleTapped: ... }`

Хочу	Пишу внутрь предмета
Долго держат	`TapHandler { onLongPressed: ... }`
Таскать пальцем	`DragHandler {}`
Увеличить двумя пальцами	`PinchHandler {}`

Пример — картинка, которую можно таскать, увеличивать и сбрасывать двойным тапом:

```
Image {
    id: photo
    source: "cat.jpg"
    width: 200; height: 200
    fillMode: Image.PreserveAspectFit

    DragHandler {}                      // таскается
    PinchHandler {}                     // увеличивается и крутится
    TapHandler {
        onDoubleTapped: { photo.scale = 1; photo.rotation = 0; photo.x = 0;
photo.y = 0 }
    }
}
```

5.6. Клавиатура закрыла поле ввода

Оберни экран в `ScrollView` — тогда его можно прокрутить, и ничего не потеряется под клавиатурой. Высота клавиатуры лежит в `Qt.inputMethod.keyboardRectangle.height`, если хочешь сдвигать что-то вручную.

5.7. Вырез под камеру и системные полоски

Иногда края экрана заняты часами, батареей или полоской жестов. В свежих версиях Qt для этого есть `SafeArea` (отступы от опасных краёв); в старых проще всего дать сверху и снизу отступы по 24–32. Точное название свойств посмотри в справке своей версии Qt (F1 в Qt Creator).

> **Сделай сам:** экран входа из 5.2. Покрути окно на компьютере, растяни его мышкой — всё должно оставаться на своих местах, а кнопка — внизу.

Урок 6. Несколько экранов и кнопка «назад»

Настоящее приложение — это несколько экранов. В Qt есть три способа между ними ходить:

Способ	Как в жизни	Пример приложения
`StackView`	Стопка бумаг: кладёшь новый лист сверху, «назад» — убираешь верхний	Почта: список писем → одно письмо
`TabBar` + `SwipeView`	Вкладки внизу, листаются свайпом	Мессенджер: Чаты / Звонки / Настройки
`Drawer`	Шторка-меню сбоку	Кнопка ☰ в углу

6.1. Стопка экранов (StackView) — полный пример

Сделаем три файла. Не забудь добавить их в `QML_FILES` в `CMakeLists.txt`.

Main.qml — главное окно со стопкой:

```
import QtQuick
import QtQuick.Controls

ApplicationWindow {
    visible: true
    width: 360; height: 640

    // верхняя панель с кнопкой «назад» и заголовком
    header: ToolBar {
        ToolButton {
            text: "‹ Назад"
            visible: stack.depth > 1          // показывать, только если есть
куда назад
            onClicked: stack.pop()            // убрать верхний лист
        }
        Label {
            anchors.centerIn: parent
            text: stack.currentItem ? stack.currentItem.title : ""
            font.bold: true
        }
    }

    StackView {
        id: stack
```

```
        anchors.fill: parent
        initialItem: "HomePage.qml"           // с какого экрана начать
    }
}
```

HomePage.qml — список товаров:

```
import QtQuick
import QtQuick.Controls

Page {
    title: "Магазин"

    ListView {
        anchors.fill: parent
        model: ["Телефон", "Наушники", "Чехол"]
        delegate: ItemDelegate {
            width: ListView.view.width
            text: modelData
            // положить в стопку новый экран и передать ему название товара
            onClicked: StackView.view.push("DetailsPage.qml", { productName:
modelData })
        }
    }
}
```

DetailsPage.qml — экран одного товара:

```
import QtQuick
import QtQuick.Controls

Page {
    property string productName: ""     // сюда придёт название
    title: productName

    Column {
        anchors.centerIn: parent
        spacing: 16
        Label { text: "Вы смотрите: " + productName; font.pixelSize: 20 }
        Button { text: "В корзину"; onClicked: console.log("Купили",
productName) }
    }
}
```

Шпаргалка по стопке:

Команда	Что делает
`stack.push("X.qml")`	Открыть экран X поверх
`stack.push("X.qml", { a: 1 })`	Открыть и передать данные
`stack.pop()`	Назад на один экран
`stack.pop(null)`	Назад до самого первого
`stack.replace("X.qml")`	Заменить текущий (например, после входа, чтобы «назад» не вернул на экран входа)
`stack.depth`	Сколько экранов в стопке

6.2. Системная кнопка «назад» на Android

Если ничего не делать, системный жест «назад» **закроет всё приложение**. Добавь в `StackView` из Main.qml:

```
StackView {
    id: stack
    anchors.fill: parent
    initialItem: "HomePage.qml"
    focus: true                                  // чтобы ловить нажатия клавиш
    Keys.onBackPressed: (event) => {
        if (stack.depth > 1) {
            stack.pop()
            event.accepted = true                // «я сам справился, не
закрывай»
        }
        // если мы на первом экране — пусть приложение закроется, это
нормально
    }
}
```

6.3. Вкладки внизу (TabBar + SwipeView)

```
import QtQuick
import QtQuick.Controls

ApplicationWindow {
    visible: true
    width: 360; height: 640

    SwipeView {
```

```
        id: pages
        anchors.fill: parent
        currentIndex: tabs.currentIndex          // синхронно с вкладками

        Page { Label { anchors.centerIn: parent; text: "Чаты" } }
        Page { Label { anchors.centerIn: parent; text: "Звонки" } }
        Page { Label { anchors.centerIn: parent; text: "Настройки" } }
    }

    footer: TabBar {
        id: tabs
        currentIndex: pages.currentIndex         // и наоборот
        TabButton { text: "Чаты" }
        TabButton { text: "Звонки" }
        TabButton { text: "Настройки" }
    }
}
```

Тыкаешь по вкладке — листается страница. Свайпаешь страницу — переключается вкладка.

6.4. Боковое меню (Drawer)

```
Drawer {
    id: menu
    width: parent.width * 0.7      // 70% ширины экрана
    height: parent.height

    Column {
        anchors.fill: parent
        ItemDelegate { width: parent.width; text: "Профиль";   onClicked:
menu.close() }
        ItemDelegate { width: parent.width; text: "Настройки"; onClicked:
menu.close() }
        ItemDelegate { width: parent.width; text: "Выйти";     onClicked:
Qt.quit() }
    }
}

// кнопка, которая его открывает
ToolButton { text: "☰"; onClicked: menu.open() }
```

Меню также открывается свайпом от левого края.

6.5. Всплывающий вопрос (Dialog)

```
Dialog {
    id: confirm
    title: "Удалить?"
    anchors.centerIn: parent
    modal: true                                    // затемнить фон
    standardButtons: Dialog.Yes | Dialog.No
    onAccepted: console.log("Удаляем")
    onRejected: console.log("Отмена")
}

Button { text: "Удалить"; onClicked: confirm.open() }
```

> **Сделай сам:** собери приложение «Заметки»: на первом экране список заметок и кнопка «+», которая через `push` открывает экран с полем ввода. Системный «назад» должен возвращать к списку.

Урок 7. «Мозги» на C++ и как они говорят с экраном

Когда логика становится сложной (правила игры, расчёты, работа с файлами), её пишут на C++. QML остаётся «лицом» и просто спрашивает «мозги».

Как они общаются — три способа:

В C++ пишем	Что это даёт в QML	Как в жизни
`Q_PROPERTY`	Свойство, которое можно читать и на которое можно привязаться	Табло со счётом
`Q_INVOKABLE`	Функция, которую QML может вызвать	Кнопка вызова официанта
`signals`	Сообщение из C++ в QML «что-то произошло»	Официант кричит «заказ готов!»

7.1. Пример полностью: крестики-нолики

«Мозги» знают поле, чей ход и кто выиграл. «Лицо» рисует клетки и передаёт нажатия.

game.h — описание «мозгов»:

```
#pragma once
#include <QObject>
#include <QStringList>
#include <QtQml/qqmlregistration.h>

class Game : public QObject
{
    Q_OBJECT
    QML_ELEMENT   // «покажи этот класс в QML под именем Game»

    // свойства: что QML может читать
    Q_PROPERTY(QStringList cells READ cells NOTIFY changed)      // 9 клеток:
"", "X" или "O"
    Q_PROPERTY(QString turn READ turn NOTIFY changed)            // чей ход
    Q_PROPERTY(QString winner READ winner NOTIFY changed)        // кто
выиграл

public:
    explicit Game(QObject *parent = nullptr);

    QStringList cells() const { return m_cells; }
    QString turn() const { return m_turn; }
    QString winner() const { return m_winner; }

    // функции, которые QML может вызвать
    Q_INVOKABLE void play(int index);
    Q_INVOKABLE void restart();

signals:
    void changed();   // «что-то поменялось, перерисуйся»

private:
    QString checkWinner() const;
    QStringList m_cells;
    QString m_turn = "X";
    QString m_winner;
};
```

game.cpp — сами «мозги»:

```
#include "game.h"

Game::Game(QObject *parent) : QObject(parent)
{
    restart();
}
```

```
void Game::restart()
{
    m_cells = QStringList(9, "");   // 9 пустых клеток
    m_turn = "X";
    m_winner.clear();
    emit changed();                 // сказать экрану: обновись
}

void Game::play(int index)
{
    // нельзя ходить: игра окончена, нет такой клетки или она занята
    if (!m_winner.isEmpty() || index < 0 || index > 8 ||
!m_cells[index].isEmpty())
        return;

    m_cells[index] = m_turn;
    m_winner = checkWinner();
    m_turn = (m_turn == "X") ? "O" : "X";
    emit changed();
}

QString Game::checkWinner() const
{
    // все выигрышные линии: 3 строки, 3 столбца, 2 диагонали
    static const int lines[8][3] = {
        {0,1,2},{3,4,5},{6,7,8},
        {0,3,6},{1,4,7},{2,5,8},
        {0,4,8},{2,4,6}
    };
    for (const auto &l : lines) {
        const QString &a = m_cells[l[0]];
        if (!a.isEmpty() && a == m_cells[l[1]] && a == m_cells[l[2]])
            return a;                       // "X" или "O"
    }
    if (!m_cells.contains(""))
        return "Ничья";                    // клеток не осталось
    return {};                              // игра продолжается
}
```

CMakeLists.txt — добавляем файлы в модуль (строчка `SOURCES`):

```
qt_add_qml_module(appcounter
    URI counter
    VERSION 1.0
    QML_FILES Main.qml
    SOURCES game.h game.cpp
)
```

Main.qml — «лицо»:

```
import QtQuick
import QtQuick.Controls
import counter                    // имя из URI в CMake — там живёт Game

ApplicationWindow {
    visible: true
    width: 360; height: 640

    Game { id: game }             // создаём «мозги»

    Column {
        anchors.centerIn: parent
        spacing: 20

        Label {
            anchors.horizontalCenter: parent.horizontalCenter
            font.pixelSize: 24
            text: game.winner === "" ? "Ходит: " + game.turn
                 : game.winner === "Ничья" ? "Ничья!"
                 : "Победил: " + game.winner
        }

        Grid {
            columns: 3
            spacing: 6
            Repeater {
                model: 9                  // 9 клеток
                delegate: Button {
                    width: 96; height: 96
                    font.pixelSize: 40
                    text: game.cells[index] // что в этой клетке
                    onClicked: game.play(index)
                }
            }
        }

        Button {
```

```
            anchors.horizontalCenter: parent.horizontalCenter
            text: "Заново"
            onClicked: game.restart()
        }
    }
}
```

Как это работает по шагам:

1. Ты тыкаешь клетку → QML вызывает `game.play(index)`.
2. C++ ставит X или O, проверяет победу и кричит `emit changed()`.
3. QML слышит сигнал и сам перерисовывает клетки и надпись (привязки!).

7.2. Главные правила

- **Забыл `emit changed()` — экран не обновится.** Это ошибка номер один.
- После добавления нового C++-класса сделай **Build → Run CMake**, иначе QML его «не увидит».
- Имя класса с большой буквы, и над ним обязательно `Q_OBJECT` и `QML_ELEMENT`.
- Долгие дела (скачать файл, посчитать что-то огромное) нельзя делать «в лоб» — приложение зависнет. Для сети в Qt уже есть готовые «невисячие» способы (урок 9).

7.3. Если нужен один общий объект на всё приложение

Например, «текущий пользователь» или «корзина». Допиши под `QML_ELEMENT` ещё строчку `QML_SINGLETON` — и в QML обращайся к нему прямо по имени класса, без создания: `Cart.add("Хлеб")`. Один объект на все экраны.

> **Сделай сам:** добавь в `Game` счёт побед (`Q_PROPERTY(int xWins ...)` и `oWins`) и покажи его над полем.

Урок 8. Работа с телефоном: разрешения, камера, GPS, датчики

Телефон не даст приложению камеру или местоположение просто так. Сначала нужно **спросить разрешение** — то самое окошко «Разрешить приложению доступ к камере?».

8.1. Разрешения

В Qt 6.5+ есть готовые предметы-разрешения прямо в QML:

```
import QtCore

CameraPermission {
    id: camPerm
}

Button {
    text: "Открыть камеру"
    onClicked: {
        if (camPerm.status === Qt.PermissionStatus.Granted)
            stack.push("CameraPage.qml")
        else
            camPerm.request()       // показать системное окошко
    }
}
```

Есть также `LocationPermission`, `MicrophonePermission`, `BluetoothPermission`, `ContactsPermission`, `CalendarPermission`.

На iPhone дополнительно нужно написать, *зачем* тебе камера, в файле `Info.plist` (иначе приложение просто упадёт):

```
<key>NSCameraUsageDescription</key>
<string>Нужна, чтобы сфотографировать аватарку</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>Нужно, чтобы показать магазины рядом</string>
```

На Android Qt обычно сам дописывает нужные строчки в `AndroidManifest.xml` по модулям, которые ты подключил. Если нет — см. урок 10, там как свой манифест сделать.

8.2. Камера: показать картинку и сфотографировать

В CMake добавь модуль `Multimedia`: в `find_package` — `Multimedia`, в `target_link_libraries` — `Qt6::Multimedia`.

CameraPage.qml:

```
import QtQuick
import QtQuick.Controls
import QtMultimedia

Page {
```

```
    title: "Камера"

    CaptureSession {
        camera: Camera { active: true }              // включить камеру
        imageCapture: ImageCapture {
            id: shot
            onImageSaved: (id, path) => {
                console.log("Фото сохранено:", path)
                preview.source = "file://" + path
            }
        }
        videoOutput: viewfinder                       // куда показывать
картинку
    }

    VideoOutput {
        id: viewfinder
        anchors.fill: parent
        fillMode: VideoOutput.PreserveAspectCrop
    }

    RoundButton {
        anchors.bottom: parent.bottom
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.bottomMargin: 32
        width: 72; height: 72
        text: "●"
        onClicked: shot.captureToFile("")             // "" = сохранить в
папку по умолчанию
    }

    Image {
        id: preview
        width: 80; height: 80
        anchors.left: parent.left; anchors.bottom: parent.bottom
        anchors.margins: 16
        fillMode: Image.PreserveAspectCrop
    }
}
```

8.3. Где я? (GPS)

Модуль `Positioning` (в CMake: `Positioning` и `Qt6::Positioning`).

```
import QtQuick
import QtQuick.Controls
import QtPositioning

Column {
    PositionSource {
        id: gps
        active: true                    // начать определять место
        updateInterval: 5000            // раз в 5 секунд
    }

    Label {
        text: gps.position.coordinate.isValid
              ? "Широта: " + gps.position.coordinate.latitude.toFixed(5)
                + "\nДолгота: " +
gps.position.coordinate.longitude.toFixed(5)
              : "Ищу спутники…"
    }
}
```

Не забудь сначала получить `LocationPermission` (8.1). В помещении GPS может долго искать спутники — это не ошибка.

8.4. Датчики: наклон телефона

Модуль `Sensors` (в CMake: `Sensors` и `Qt6::Sensors`). Пример — шарик, который катается, если наклонять телефон:

```
import QtQuick
import QtSensors

Item {
    anchors.fill: parent

    Accelerometer {
        id: accel
        active: true
        onReadingChanged: {
            // чем сильнее наклон, тем быстрее едет шарик
            ball.x = Math.max(0, Math.min(parent.width - ball.width,  ball.x
- reading.x * 2))
            ball.y = Math.max(0, Math.min(parent.height - ball.height, ball.y
+ reading.y * 2))
        }
    }

    Rectangle { id: ball; width: 50; height: 50; radius: 25; color: "tomato"
}
}
```

8.5. Вибрация, ссылки, «поделиться»

- Открыть сайт, позвонить или написать письмо — одной строчкой:

```
Button { text: "Сайт";      onClicked: Qt.openUrlExternally("https://qt.io")
}
Button { text: "Позвонить"; onClicked:
Qt.openUrlExternally("tel:+79990000000") }
Button { text: "Написать";  onClicked:
Qt.openUrlExternally("mailto:test@example.com") }
```

- Вибрация и системное меню «Поделиться» в чистом Qt готовыми не идут: делаются через вызов Android (Java/Kotlin) из C++ — через `QJniObject`. Это уже продвинутая тема, на старте можно обойтись без него.

8.6. Приложение свернули

Когда человек сворачивает приложение, телефон может его «усыпить» или даже закрыть. Поэтому **сохраняй данные в момент сворачивания** и выключай камеру/GPS, чтобы не жрать батарею:

```
Connections {
    target: Qt.application
    function onStateChanged() {
        if (Qt.application.state === Qt.ApplicationSuspended
            || Qt.application.state === Qt.ApplicationInactive) {
            console.log("Свернули — сохраняю данные")
            gps.active = false
        } else if (Qt.application.state === Qt.ApplicationActive) {
            console.log("Вернулись")
            gps.active = true
        }
    }
}
```

> **Сделай сам:** экран с кнопкой «Где я?». По нажатию — спроси разрешение, включи GPS и покажи координаты. Вторая кнопка открывает карту в браузере: `Qt.openUrlExternally("https://maps.google.com/?q=" + lat + "," + lon)`.

Урок 9. Сохраняем данные и ходим в интернет

Без сохранения всё, что человек навводил, **исчезает после закрытия приложения**. Выбирай способ по объёму:

Что храним	Способ	Сложность
Настройки, имя, рекорд, тёмная тема	`Settings`	Очень просто
Небольшой список (заметки, покупки)	`Settings` + JSON	Просто
Тысячи записей, поиск, сортировка	База SQLite	Средне

9.1. Settings — запомнить что-нибудь

Работает как записная книжка: свойства **сами сохраняются и сами загружаются** при следующем запуске.

```
import QtQuick
import QtQuick.Controls
import QtCore

ApplicationWindow {
    visible: true
```

```
    width: 360; height: 640

    Settings {
        id: saved
        property string userName: ""     // значение по умолчанию
        property int bestScore: 0
        property bool darkTheme: false
    }

    Column {
        anchors.centerIn: parent
        spacing: 12
        TextField {
            text: saved.userName
            placeholderText: "Твоё имя"
            onTextEdited: saved.userName = text   // написал — сразу
запомнилось
        }
        Switch {
            text: "Тёмная тема"
            checked: saved.darkTheme
            onToggled: saved.darkTheme = checked
        }
        Label { text: "Рекорд: " + saved.bestScore }
    }
}
```

Закрой приложение, открой снова — имя и галочка на месте.

> Важно: для `Settings` в `main.cpp` должны быть заданы имя приложения и организации. Допиши перед созданием `engine`:

```
app.setOrganizationName("MyCompany");
app.setApplicationName("MyApp");
```

9.2. Список, который не пропадает (Settings + JSON)

JSON — это способ превратить список в строчку текста и обратно. Строчку легко сохранить.

```
Settings { id: store; property string notesJson: "[]" }

ListModel { id: notes }

// при запуске — достаём список из строчки
```

```
Component.onCompleted: {
    const list = JSON.parse(store.notesJson)
    for (const n of list) notes.append(n)
}

// вызывай после каждого изменения
function save() {
    const list = []
    for (let i = 0; i < notes.count; i++) list.push(notes.get(i))
    store.notesJson = JSON.stringify(list)
}

function addNote(text) {
    notes.append({ text: text, done: false })
    save()
}
```

9.3. База данных (когда данных много)

База — это таблица, как в Excel, но внутри телефона. Работаем из C++ (в CMake: `Sql` и `Qt6::Sql`):

```
#include <QSqlDatabase>
#include <QSqlQuery>
#include <QStandardPaths>
#include <QDir>

void openDb()
{
    // папка, куда приложению РАЗРЕШЕНО писать на телефоне
    const QString dir =
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
    QDir().mkpath(dir);

    QSqlDatabase db = QSqlDatabase::addDatabase("QSQLITE");
    db.setDatabaseName(dir + "/notes.db");
    db.open();

    QSqlQuery q;
    q.exec("CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY, text
TEXT)");
}

void addNote(const QString &text)
{
    QSqlQuery q;
```

```
    q.prepare("INSERT INTO notes (text) VALUES (?)");   // ? — место для
данных
    q.addBindValue(text);
    q.exec();
}
```

Дальше оборачиваешь это в класс с `Q_INVOKABLE`, как в уроке 7. **Никогда не склеивай текст от пользователя прямо в запрос** — только через `?` и `addBindValue`, иначе сломается на первой же кавычке.

9.4. Интернет: скачать данные с сайта

Самый простой способ — прямо из QML через `XMLHttpRequest` (несмотря на название, отлично работает с JSON). Он не подвешивает экран.

Пример: погода в Москве с бесплатного сервиса open-meteo (ключ не нужен):

```
import QtQuick
import QtQuick.Controls

ApplicationWindow {
    visible: true
    width: 360; height: 640

    function loadWeather() {
        status.text = "Загружаю…"
        const xhr = new XMLHttpRequest()
        xhr.onreadystatechange = function() {
            if (xhr.readyState !== XMLHttpRequest.DONE) return   // ещё не
докачалось
            if (xhr.status === 200) {
                const data = JSON.parse(xhr.responseText)
                status.text = "Сейчас " + data.current_weather.temperature +
" °C"
            } else {
                status.text = "Ошибка: нет интернета?"
            }
        }
        xhr.open("GET", "https://api.open-meteo.com/v1/forecast?
latitude=55.75&longitude=37.62&current_weather=true")
        xhr.send()
    }

    Column {
        anchors.centerIn: parent
        spacing: 16
```

```
        Label { id: status; text: "Нажми кнопку"; font.pixelSize: 24 }
        Button { text: "Узнать погоду"; onClicked: loadWeather() }
    }
}
```

Разбор:

1. `open("GET", адрес)` — «хочу получить данные вот отсюда».
2. `send()` — отправить. Программа **не ждёт**, а идёт дальше.
3. Когда ответ придёт, сработает `onreadystatechange`.
4. `status === 200` — всё хорошо. Другие числа — ошибка (404 — не найдено, 0 — нет связи).
5. `JSON.parse` превращает ответ в объект, из которого достаём температуру через точку.

Отправить данные на сервер (POST):

```
xhr.open("POST", "https://example.com/api/notes")
xhr.setRequestHeader("Content-Type", "application/json")
xhr.send(JSON.stringify({ text: "Купить хлеб" }))
```

Для больших проектов сеть делают в C++ через `QNetworkAccessManager` (модуль `Network`) — принцип тот же: отправил запрос, получил сигнал `finished`, когда ответ пришёл.

> На Android для интернета нужно разрешение `INTERNET` в манифесте (Qt обычно добавляет его сам). Адреса с `http://` без **s** на современных телефонах по умолчанию запрещены — используй `https://`.

> **Сделай сам:** вместо Москвы возьми координаты из GPS (урок 8.3) и подставь их в адрес. А последнюю температуру сохраняй в `Settings`, чтобы показывать её без интернета.

Урок 10. Собираем и выкладываем в магазин

До сих пор ты собирал «черновую» версию (Debug). Для людей нужна **чистовая (Release)** — она быстрее, меньше и **подписана**. Подпись — это как печать на документе: доказывает, что приложение сделал именно ты.

10.1. Иконка, название, версия (Android)

1. Открой вкладку **Projects** (слева) → kit Android → **Build → Build Steps → Build Android APK**.
2. Нажми **Create Templates**. В проекте появится папка `android` с файлом `AndroidManifest.xml`.
3. Открой этот файл — Qt Creator покажет удобный редактор: название приложения, **Package name** (уникальный адрес вида `ru.vasya.notes` — потом поменять нельзя!), иконки, разрешения.
4. В `CMakeLists.txt` укажи папку и версию:

```
set_target_properties(appnotes PROPERTIES
    QT_ANDROID_PACKAGE_SOURCE_DIR "${CMAKE_CURRENT_SOURCE_DIR}/android"
    QT_ANDROID_VERSION_CODE 1          # целое число, +1 при каждой выкладке
    QT_ANDROID_VERSION_NAME "1.0"      # то, что видит человек
)
```

10.2. Создаём ключ для подписи (один раз на всю жизнь)

1. В том же месте (**Build Android APK**) раскрой **Application Signature**.
2. **Keystore → Create…** Заполни пароль, имя, город. Нажми Save — получишь файл `.keystore`.
3. Поставь галочку **Sign package**.

> ⚠️ **Самое важное в этом уроке.** Сохрани файл `.keystore` и пароль в надёжном месте (облако + флешка). Потерял — в большинстве случаев больше не сможешь выпускать обновления. Никому не отдавай и не клади в публичный GitHub.

10.3. Собираем файл

1. Внизу слева (значок монитора) переключи **Debug → Release**.
2. В Build Android APK отметь нужное:

Хочу	Ставлю	Получаю
Скинуть другу поиграть	Ничего не менять	Файл `.apk`
Выложить в Google Play	**Build Android App Bundle (*.aab)**	Файл `.aab`

3. **Build → Build Project** (Ctrl+B). Готовый файл лежит в папке сборки, путь виден внизу в панели **Compile Output** (ищи слова `.apk` или `.aab`).

`.apk` можно просто переслать в мессенджере — друг откроет и установит (телефон спросит разрешение «устанавливать из неизвестных источников»).

10.4. Google Play

1. Зарегистрируйся в **Google Play Console** (play.google.com/console). Регистрация платная, разовая — актуальную цену смотри на сайте.
2. **Create app** → название, язык, бесплатное/платное.
3. Заполни карточку: описание, иконка 512×512, минимум 2 скриншота, политика конфиденциальности (ссылка на страничку с текстом), анкета про возрастной рейтинг.
4. **Testing → Internal testing** → загрузи свой `.aab`. Сначала проверь на себе и друзьях.
5. Потом **Production → Create release** → отправь на проверку. Проверка идёт от нескольких часов до нескольких дней.

У новых личных аккаунтов Google может потребовать закрытое тестирование с несколькими тестерами перед выкладкой — точные требования Play Console покажет сам.

Обновление: увеличь `QT_ANDROID_VERSION_CODE` на 1, собери с тем же ключом, загрузи новый `.aab`.

10.5. iPhone и App Store

1. Нужен платный **Apple Developer Program** (ежегодная подписка). Бесплатного Apple ID хватает только чтобы поставить на свой телефон на время.
2. В Qt Creator выбери kit **iOS**, режим **Release**, собери. Получится проект Xcode.
3. Открой его в **Xcode** (файл `.xcodeproj` в папке сборки). Во вкладке **Signing & Capabilities** выбери свою команду и уникальный **Bundle Identifier**.
4. **Product → Archive** → **Distribute App** → **App Store Connect**. Дальше карточка и проверка на сайте App Store Connect, по тому же принципу, что у Google. Для теста на друзьях есть **TestFlight**.

> **Сделай сам:** собери подписанный Release `.apk` своих крестиков-ноликов с собственной иконкой и поставь его на чужой телефон.

Урок 11. Когда что-то сломалось или тормозит

11.1. Как искать ошибку — алгоритм

1. **Посмотри вниз Qt Creator**: панели **Issues** (ошибки сборки) и **Application Output** (что пишет работающее приложение). В 90% случаев ответ там.
2. **Читай первую красную строчку**, а не последнюю. Остальные часто — последствия первой.
3. В строчке ошибки есть имя файла и номер строки (`Main.qml:42`). Щёлкни — попадёшь туда.
4. Не понятно, что происходит? Расставь `console.log("тут 1", переменная)` и смотри, до какого места доходит.
5. Скопируй текст ошибки в поисковик — с ней почти наверняка уже кто-то сталкивался.

Отладчик (по шагам). Щёлкни слева от номера строки — появится красная точка. Запусти не ▶, а ▶ с жучком (F5). Программа остановится на точке, и ты увидишь значения всех переменных. F10 — следующая строка.

11.2. Частые проблемы и решения

Симптом	Причина	Решение
На компьютере работает, на телефоне чёрный экран	Файл не добавлен в `QML_FILES`	Допиши в CMakeLists.txt, Run CMake
Картинка не показывается на телефоне	Картинка не вшита в приложение	Добавь в `qt_add_qml_module` строчку `RESOURCES cat.png`
`X is not a type`	Нет нужного `import` или модуля в CMake	Проверь import и `find_package`
`ReferenceError: x is not defined`	Опечатка в `id` или имени	Регистр букв важен: `myBtn` ≠ `mybtn`
Экран не обновляется после C++	Забыл `emit ...Changed()`	Добавь emit
Приложение вылетает на iPhone при открытии камеры	Нет описания в Info.plist	Урок 8.1

Симптом	Причина	Решение
Интерфейс замирает на пару секунд	Долгая работа «в лоб»	Сеть — асинхронно (9.4), тяжёлое — в C++ в отдельном потоке
Сборка падает со странными ошибками про пути	Русские буквы или пробелы в пути к проекту	Перенеси проект в `C:\qt_projects`
Всё сломалось непонятно почему	Старые файлы сборки	**Build → Clean**, потом собери заново

11.3. Чтобы не тормозило

- **Не делай огромные картинки.** Фото 4000×3000 для иконки 48×48 — жрёт память. Уменьшай заранее или пиши `sourceSize.width: 96`.
- **Длинные списки — только через `ListView`**, не через `Column` + `Repeater`. ListView создаёт только те строки, что видны на экране.
- **Не грузи всё сразу.** Тяжёлые экраны оборачивай в `Loader { active: нужен_ли; source: "Heavy.qml" }` — он создаст экран, только когда надо.
- **Прячь правильно.** Невидимые штуки делай `visible: false`, а не `opacity: 0` — прозрачное всё равно рисуется.
- **Проверяй скорость на Release-сборке** и на дешёвом телефоне. Debug всегда медленнее.
- Для серьёзного анализа есть **Analyze → QML Profiler** — покажет, что именно тормозит.

Итоговый проект: «Мои заметки»

Собери всё, что узнал, в одно приложение. Иди по пунктам и ставь галочки:

- [ ] Главный экран — список заметок (`ListView` + `ListModel`, урок 4)
- [ ] Кнопка «+» открывает экран редактирования (`StackView`, урок 6)
- [ ] Вёрстка через `ColumnLayout`, нормально выглядит боком (урок 5)
- [ ] Долгое нажатие на заметку → диалог «Удалить?» (уроки 5 и 6)
- [ ] Системный «назад» работает правильно (урок 6.2)
- [ ] Заметки сохраняются после закрытия (`Settings` + JSON, урок 9.2)
- [ ] К заметке можно прикрепить фото с камеры (урок 8.2)
- [ ] Настройка «Тёмная тема», которая запоминается (урок 9.1)

- [ ] Подписанный Release `.apk` с своей иконкой (урок 10)

Подсказка про тёмную тему. В Qt есть готовый стиль Material. Подключи его и переключай тему одним свойством:

```
import QtQuick.Controls.Material

ApplicationWindow {
    Material.theme: saved.darkTheme ? Material.Dark : Material.Light
    Material.accent: Material.Teal
}
```

А в `main.cpp` перед созданием `engine` допиши `QQuickStyle::setStyle("Material");` (нужен `#include <QQuickStyle>`).

Что дальше

- **Справка прямо в Qt Creator:** поставь курсор на любое слово (например, `ListView`) и нажми **F1**. Там все свойства и примеры.
- **Готовые примеры:** на стартовом экране Qt Creator вкладка **Examples** — сотни рабочих проектов, в том числе для телефонов. Открывай и разбирай.
- **Темы для изучения потом:** свои модели на C++ (`QAbstractListModel`), уведомления, вызов Android-кода из C++ (`QJniObject`), карты (`QtLocation`), анимации и состояния (`State`, `Transition`).

> Главный секрет: **делай маленькими шагами и запускай после каждого.** Добавил кнопку — запустил. Добавил список — запустил. Тогда если что-то сломалось, ты точно знаешь, где.